

```

#include<lpc21xx.h>
#include<string.h>
#include"header.h"
char k[20];
char r[13];
char h[13]="123456789";
char *s="1234";
int t[6];
int i,f1=0,f2=0;
CAN tx;
u8 am_pm;
int main()
{
    can_ini();
    uart0_init(9600);
    uart0_tx_string("hello\r\n");
    //can_ini();
    uart0_tx_string("after can\r\n");
    i2c_init();
    uart0_tx_string("after i2c\r\n");
    //uart0_tx('a');
    while(1)
    {
        uart0_tx_string("waiting\n\r");
        if(IOPIN0&PIR)
        {
            delay_sec(1);

if(IOPIN0&PIR)
{

uart0_tx_string("motion detected\r\n");
uart0_tx_string("enter rfid\r\n");
uart0_rx_string(r,13);
uart0_tx("\n");
uart0_tx_string(s);
uart0_tx("\n");

if(strcmp(r,h)==0)
{
    f1=1;
    uart0_tx_string("successfully done");
    rtc_write();
        read(t);
        PWMMR2=1500;
        PWMLER=(1<<2);
        delay_sec(2);
        PWMMR2=1000;
        PWMLER=(1<<2);
        uart0_tx_string("door opened\r\n");
            if(t[2]<12)
                am_pm='A';

```

```

        else
            am_pm='P';

        eeprom_store_password(h);
        eeprom_store_datetime(t[0],t[1],t[2],am_pm,t[3],t[4],t[5]);
        uart0_tx_string("Data stored\r\n");
    }

else
{
    f2=1;
    uart0_tx_string("RFID card is not matched\r\n");
    uart0_tx_string("enter the key\r\n");
    for(i=0;i<4;i++)
    {
        k[i]=key();
        uart0_tx(k[i]);
    }

    uart0_tx("\n");

    if((strcmp(k,s))==0)
    {
        uart0_tx_string("password is correct\r\n");
        rtc_write();
        read(t);
        uart0_tx_string("DOOr opened\r\n");

        PWMMR2=1500;
        PWMLER=(1<<2);
        delay_sec(2);
        PWMMR2=1000;
        PWMLER=(1<<2);

        if(t[2]<12)
            am_pm='A';
        else
            am_pm='P';

        eeprom_store_password(s);
        //eeprom_store_datetime(t[0],t[1],t[2],am_pm,t[3],t[4],t[5]);
        eeprom_store_datetime(t[0],t[1],t[2],am_pm,t[3],t[4],t[5]);
        uart0_tx_string("Data stored\r\n");
    }

else
    uart0_tx_string("not correct\n");
}

tx.id=0x100;

```

```

tx.rtr=0;
tx.dlc=4;
if(f1)
{
    f1=0;
    //tx.id=0x100;
    //tx.rtr=0;
    //tx.dlc=4;
    tx.byteA=123456789 ;
    can_tx(tx);
}
else if(f2)
{
    f2=0;
    tx.byteA=1234;
    can_tx(tx);
}
else
uart0_tx_string("MOTION not detected\r\n");
}
}
}
}

```

uart

```

#include<lpc21xx.h>
#include"header.h"
void uart0_init(unsigned int baud)
{
    int pclk,result=0;
    if(VPBDIV==0)
    pclk=15000000;
    else if(VPBDIV==1)
    pclk=6000000;
    else if(VPBDIV==2)
    pclk=3000000;
    result=pclk/(16*baud);
    PINSEL0|=0x05 ;
    U0LCR=0x83;
    U0DLL=(result&0xff);
    U0DLM=(result>>8)&0xff;
    U0LCR=0x03;
}

```

```

#define THRE ((U0LSR>>5)&1)
void uart0_tx(unsigned char data)
{
    U0THR=data;
    while(THRE==0);
}

```

```

}

#define RDR (U0LSR&1)
unsigned char uart0_rx()
{
    while(RDR==0);
    return U0RBR;
}

void uart0_tx_string(char *p)
{
    while(*p!=0)
    {
        uart0_tx(*p);
        ++p;
    }
}

void uart0_binary(int num)
{
    int pos,temp;
    for(pos=7;pos>=0;pos--)
    {
        temp=num>>pos;
        uart0_tx(temp+48);
    }
}

void uart0_int(int num )
{
    if(num>10)
        uart0_int(num/10);

    uart0_tx((num%10)+48);
}

void uart0_hex(char num)
{
    char n;
    uart0_tx_string("0x");
    n=((num&0xf0)>>4);
    if(n>9)
        uart0_tx('A'+(n-10));
    else
        uart0_tx(n+48);
    n=(num&0x0f);
    if(n>9)
        uart0_tx('A'+(n-10));
    else
        uart0_tx(n+48);
}

```

```

}

void uart0_rx_string(char*p,int max)
{
int i;
for(i=0;i<max;i++)
{
while(RDR==0);
p[i]=U0RBR;
uart0_tx(p[i]);
if(p[i]=='\r' || p[i]=='\n' )
break;
}
p[i]='\0';
}

void uart0_float(float f)
{
int num;
num=f;
uart0_int(num);
num=(f-num)*1000000;
uart0_int(num);
}

void uart0_time_div(char t)
{
uart0_tx((t/16)+48);
uart0_tx((t%16)+48);
}

```

can tx

```

#include<lpc21xx.h>
#include"header.h"
void can_ini(void)
{
// VPBDIV=1;
PINSEL1|=0x40000;
C1MOD=1;
C1BTR=0x001C001D;
AFMR=2;
C1MOD=0;
//delay_ms(100);
}
#define TCS ((C2GSR>>3)&1)
void can_tx(CAN v)
{
C1TID1=v.id;
C1TFI1=v.dlc<<16;
if(v.rtr==0)
{

```

```

C1TDA1=v.byteA;
  C1TDB1=v.byteB;
}
else
{
  C1TFI1|=(1<<30);
}
  C1CMR=0x21;
  while(TCS==0);
}

```

I2C drive

```

/*i2c_driver.c*/

```

```

#include <LPC21xx.H>

```

```

#include "header.h"

```

```

void i2c_init(void){

```

```

    PINSEL0|=0x50;//P0.2->SCL, P0.3->SDA

```

```

    I2CONSET=(1<<6);//i2c interface is enabled & master mode selected

```

```

    I2SCLH=I2SCLL=75;//100Kbps freq

```

```

}

```

```

#define SI ((I2CONSET>>3)&1)

```

```

void i2c_write(u8 sa, u8 mr, u8 data){

```

```

    /*1. generate start condi*/

```

```

    I2CONSET=(1<<5);//STA=1 (generates start condi)

```

```

    I2CONCLR=(1<<3);//SI=0

```

```

    while(SI==0);//waiting for start condi to generate

```

```

    I2CONCLR=(1<<5);//STA=0*

```

```

    /*2. send sa+w & wait for ack (check status code)*/

```

```

    I2DAT=sa;//send SA+W

```

```

I2CONCLR=(1<<3); //clear SI

while(SI==0); //waiting for sa+w to transmit

if(I2STAT==0x20){

    uart0_tx_string("Err: SA+W\r\n");

    goto exit;

}

/*3. send memory addr & check ack*/

I2DAT=mr; //send m/r location

I2CONCLR=(1<<3); //clear SI

while(SI==0); //waiting for sa+w to transmit

if(I2STAT==0x30){

    uart0_tx_string("Err: Memory Addr\r\n");

    goto exit;

}

/*4. send data & wait for ack*/

I2DAT=data; //send data

I2CONCLR=(1<<3); //clear SI

while(SI==0); //waiting for sa+w to transmit

if(I2STAT==0x30){

    uart0_tx_string("Err: Data\r\n");

    goto exit;

}

/*5. generate stop condi*/

```

exit:

I2CONSET=(1<<4);//STO=1(generate stop condi)

I2CONCLR=(1<<3);//SI*

}

u8 i2c_read(u8 sa,u8 mr){

u8 temp;

/*1.generate start condi*/

I2CONSET=(1<<5);//STA=1 (generates start condi)

I2CONCLR=(1<<3);//SI=0

while(SI==0);//waiting for start condi to generate

I2CONCLR=(1<<5);//STA=0*

/*2.send sa+w & check ack*/

I2DAT=sa&0xFE;//send SA+W

I2CONCLR=(1<<3);//clear SI

while(SI==0);//waiting for sa+w to transmit

if(I2STAT==0x20){

uart0_tx_string("Err: SA+W\r\n");

goto exit;

}

/*3.send memory addr & check ack*/

I2DAT=mr;//send m/r location

I2CONCLR=(1<<3);//clear SI

while(SI==0);//waiting for sa+w to transmit

if(I2STAT==0x30){


```

        uart0_tx_string("Err: Memory Addr\r\n");

        goto exit;
    }

/*4.generate restart*/

I2CONSET=(1<<5);//STA=1 (generates start condi)

I2CONCLR=(1<<3);//SI=0

while(SI==0);//waiting for start condi to generate

I2CONCLR=(1<<5);//STA=0*

/*5.send sa+r & check ack*/

I2DAT=sa;//send sa+r

I2CONCLR=(1<<3);//clear SI

while(SI==0);

if(I2STAT==0x48){

    uart0_tx_string("Err: SA+R\r\n");

    goto exit;

}

/*6.read data & send noack*/

I2CONCLR=(1<<3);//clear SI

while(SI==0);//waiting for data to receive

temp=I2DAT;

/*7.generate stop condi*/

exit:

```

```
I2CONSET=(1<<4);//STO=1(generate stop condi)
```

```
I2CONCLR=(1<<3);//SI*
```

```
/*8.return received data*/
```

```
return temp;
```

```
}
```

```
rtc
```

```
#include<lpc21xx.h>
```

```
#include"header.h"
```

```
extern u8 h,m,s,D,M,Y;
```

```
void rtc_write()
```

```
{
```

```
i2c_init();
```

```
uart0_init(9600);
```

```
//    uart0_tx_string("ext rtc testing\r\n");
```

```
/*Set 11:59:55 PM in rtc*/
```

```
i2c_write(0xD0,0x2,0x23);//set hrs
```

```
i2c_write(0xD0,0x1,0x59);//set mins
```

```
i2c_write(0xD0,0x0,0x55);//set secs
```

```
i2c_write(0xD0,0x4,0x11);//set secs
```

```
i2c_write(0xD0,0x5,0x08);//set secs
```

```
i2c_write(0xD0,0x6,0x26);//set secs
```

```
}
```

```
/*read & display rtc time on lcd*/
```

```
void read(int a[])
```

```
{
```

```
a[0]=i2c_read(0xD1,0x2);//read hrs
```

```
a[1]=i2c_read(0xD1,0x1);//read mins
```

```
a[2]=i2c_read(0xD1,0x0); //read secs
```

```
a[3]=i2c_read(0xD1,0x4); //read hrs
```

```
a[4]=i2c_read(0xD1,0x5); //read mins
```

```
a[5]=i2c_read(0xD1,0x6);
```

```
//am_pm=(h>>5)&1;
```

```
uart0_tx((a[0]/16)+48);
```

```
uart0_tx((a[0]%16)+48);
```

```
uart0_tx(':');
```

```
uart0_tx((a[1]/16)+48);
```

```
uart0_tx((a[1]%16)+48);
```

```
uart0_tx(':');
```

```
uart0_tx((a[2]/16)+48);
```

```
uart0_tx((a[2]%16)+48);
```

```
uart0_tx("\r");
```

```
uart0_tx((a[3]/16)+48);
```

```
uart0_tx((a[3]%16)+48);
```

```
uart0_tx(':');
```

```
uart0_tx((a[4]/16)+48);
```

```
uart0_tx((a[4]%16)+48);
```

```
uart0_tx(':');
```

```
uart0_tx((a[5]/16)+48);
```

```
uart0_tx((a[5]%16)+48);
```

```
uart0_tx("\r");
```

```

//          if(flag==1){

//          flag=0;

//          //do t & d settings

        }

```

EepROM

```

#include<lpc21xx.h>
#include"header.h"
void eeprom_store_password(char *password)
{
    u8 i;
    for(i=0; password[i]!='\0'; i++)
    {
        i2c_write(0xA0,i,password[i]);
        delay_ms(10);
    }
    i2c_write(0xA0,i,'\0');
    delay_ms(10);
}

void eeprom_store_datetime(u8 sec,u8 min,u8 hour,u8 am_pm,
u8 date,u8 month,u8 year)
{
    i2c_write(0xA0,0x10,sec); delay_ms(10);
    i2c_write(0xA0,0x11,min); delay_ms(10);
    i2c_write(0xA0,0x12,hour); delay_ms(10);
    i2c_write(0xA0,0x13,am_pm); delay_ms(10);
    i2c_write(0xA0,0x14,date); delay_ms(10);
    i2c_write(0xA0,0x15,month); delay_ms(10);
    i2c_write(0xA0,0x16,year); delay_ms(10);
}

```